

```
In [1]: import json
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
```

Loading the data

To get started let's first load in the data! We will make use of the helper functions provided in `data_utils.py`

```
In [2]: from data_utils import get_images, get_labels
```

The following is the list of disasters in our dataset.

```
In [3]: disaster_list = ["socal-fire", "midwest-flooding", "hurricane-matthew"]
```

We will load each disaster train dataset into a dictionary. Recall that each disaster consists of images and labels (0 - 3) of the damage level.

```
In [4]: data = {}
split = "train"
with open('config.json') as config_file:
    config = json.load(config_file)
    data_dir = config['data_dir']

    for disaster in disaster_list:
        print(f"Loading {split} images and labels for {disaster} dataset...")
        images = get_images(data_dir, disaster, split=split)
        labels = get_labels(data_dir, disaster, split=split)
        data[disaster] = {"images": images, "labels": labels}
```

Loading train images and labels for socal-fire dataset...

Loading train images and labels for midwest-flooding dataset...

Loading train images and labels for hurricane-matthew dataset...

Let's visualize an image and its label in the dataset.

```
In [5]: disaster = disaster_list[0]
images = data[disaster]["images"]
labels = data[disaster]["labels"]

# index of the image to display
idx = 5
img = images[idx]
label = labels[idx]

plt.title(f"{disaster}, image {idx}, label {label}")
plt.imshow(img.astype(np.uint8))
plt.axis("off")
```

```
plt.show()

#
print(f"image shape: {img.shape}")
```

socal-fire, image 5, label 3

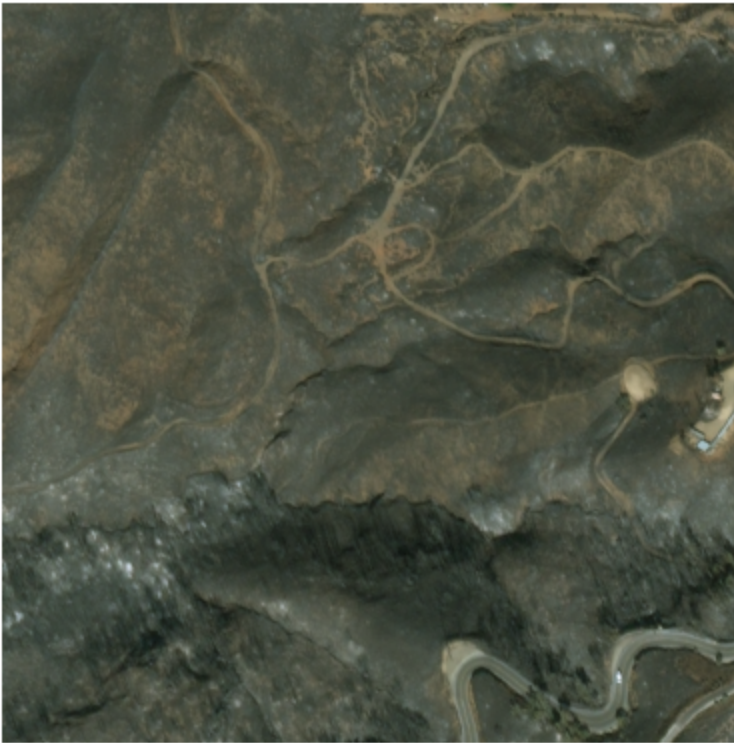


image shape: (1024, 1024, 3)

Extracting Color Information

Each image is an RGB image represented as a numpy array of dimensions (width, height, 3). The last dimension corresponds to the RGB *color channels* in the image. We can split the image by color channels and visualize each of them individually.

```
In [6]: plt.figure(figsize=(12, 12))

plt.subplot(2, 2, 1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")

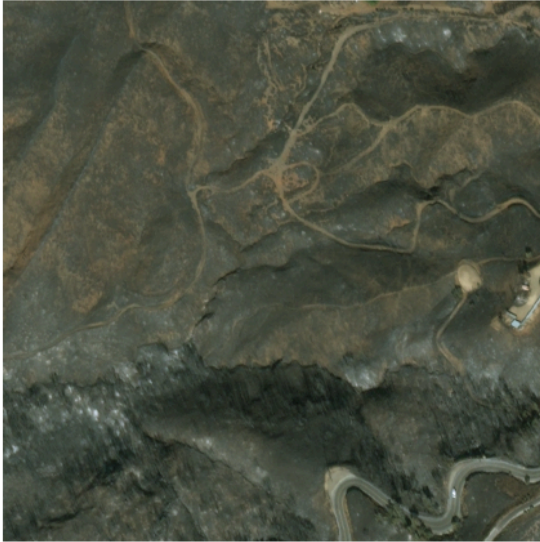
plt.subplot(2, 2, 2)
plt.imshow(img[:, :, 0], cmap="Reds", vmin=0, vmax=255)
plt.title("Red Channel")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(img[:, :, 1], cmap="Greens", vmin=0, vmax=255)
plt.title("Green Channel")
plt.axis("off")
```

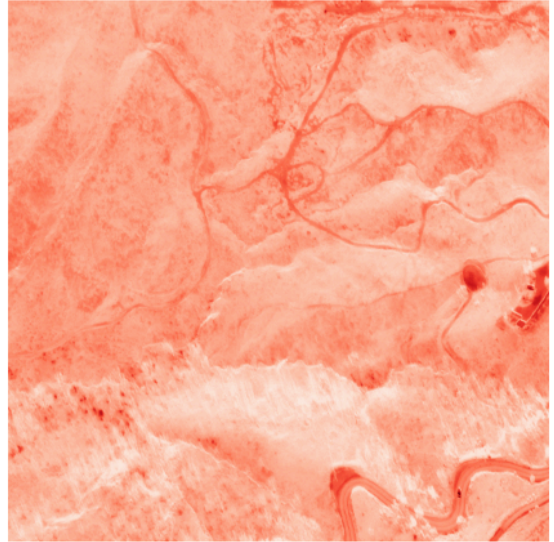
```
plt.subplot(2, 2, 4)
plt.imshow(img[:, :, 2], cmap="Blues", vmin=0, vmax=255)
plt.title("Blue Channel")
plt.axis("off")

plt.show()
```

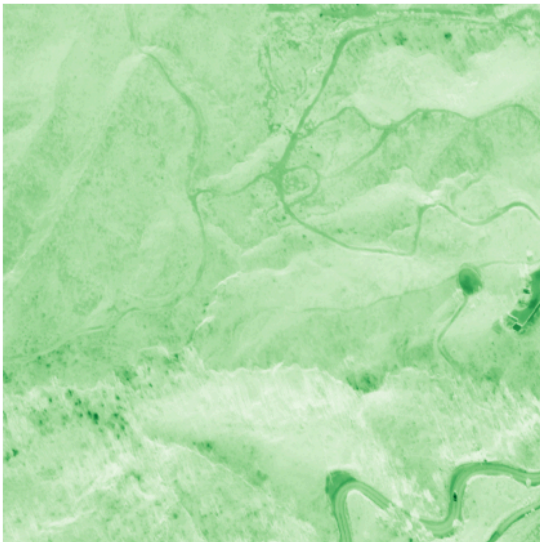
Original Image



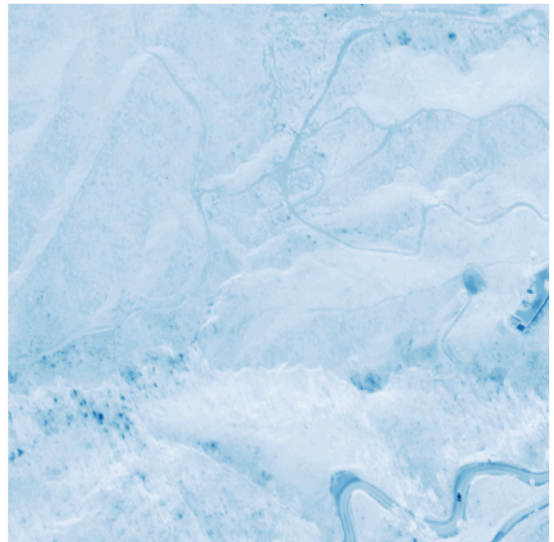
Red Channel



Green Channel



Blue Channel



We can zoom in on just a pixel as well.

```
In [7]: coord = (15, 35)
pixel_value = img[coord[0], coord[1], :]
pixel_image = np.array([[pixel_value]])

plt.imshow(pixel_image)
plt.axis("off")
plt.show()
```

```
print(f"Pixel value: {pixel_value}, coordinate: {coord}")
```



Pixel value: [66 73 60], coordinate: (15, 35)

Extracting Edge and Texture Information

So far we have considered the **color** information in an image. However, other important aspects of an image are the edges and texture.

Let us consider some popular feature processing methods for extracting edge and texture information. We will use the utilities provided in `feature_utils.py`.

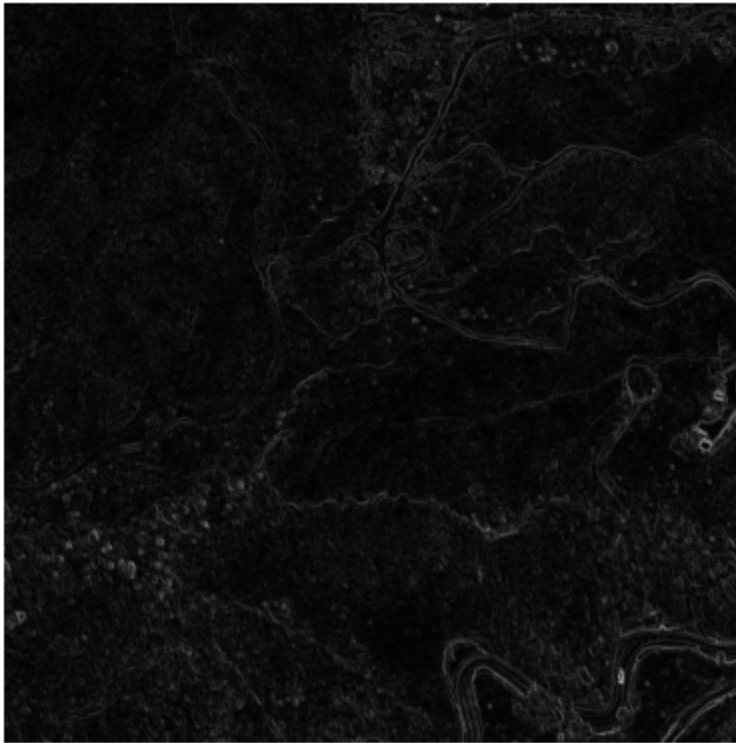
```
In [173... from feature_utils import get_sobel_features, get_gabor_features, generate_g
```

Let's try the Sobel Edge Filter.

Read more about Sobel edge detection: https://en.wikipedia.org/wiki/Sobel_operator and https://scikit-image.org/docs/stable/auto_examples/edges/plot_edge_filter.html

```
In [174... edges = get_sobel_features(img)
plt.imshow(edges, cmap="gray")
plt.title("Sobel Edge Detection")
plt.axis("off")
plt.show()
```

Sobel Edge Detection

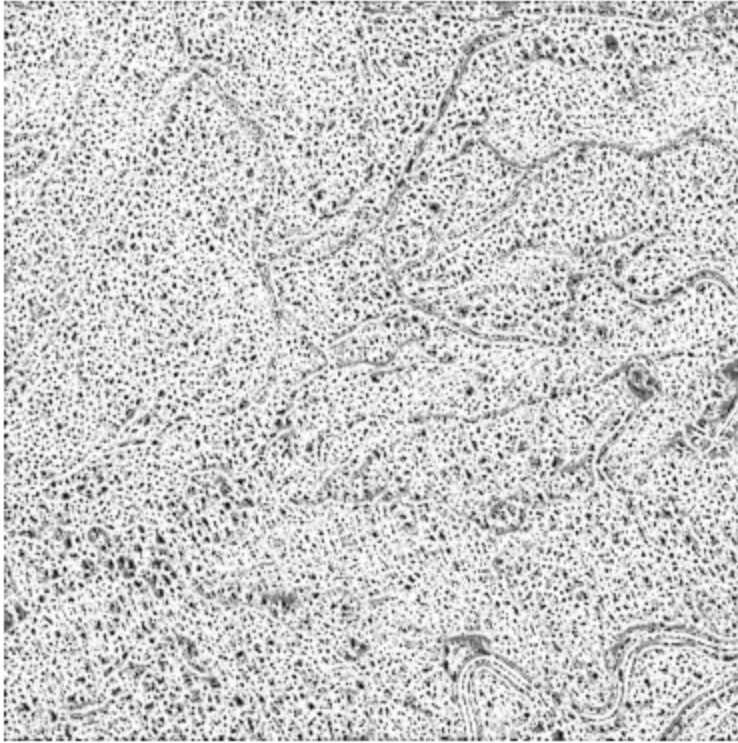


Now let's look at local binary patterns.

Read more about LBP: https://en.wikipedia.org/wiki/Local_binary_patterns and https://scikit-image.org/docs/stable/auto_examples/features_detection/plot_local_binary_pattern.html

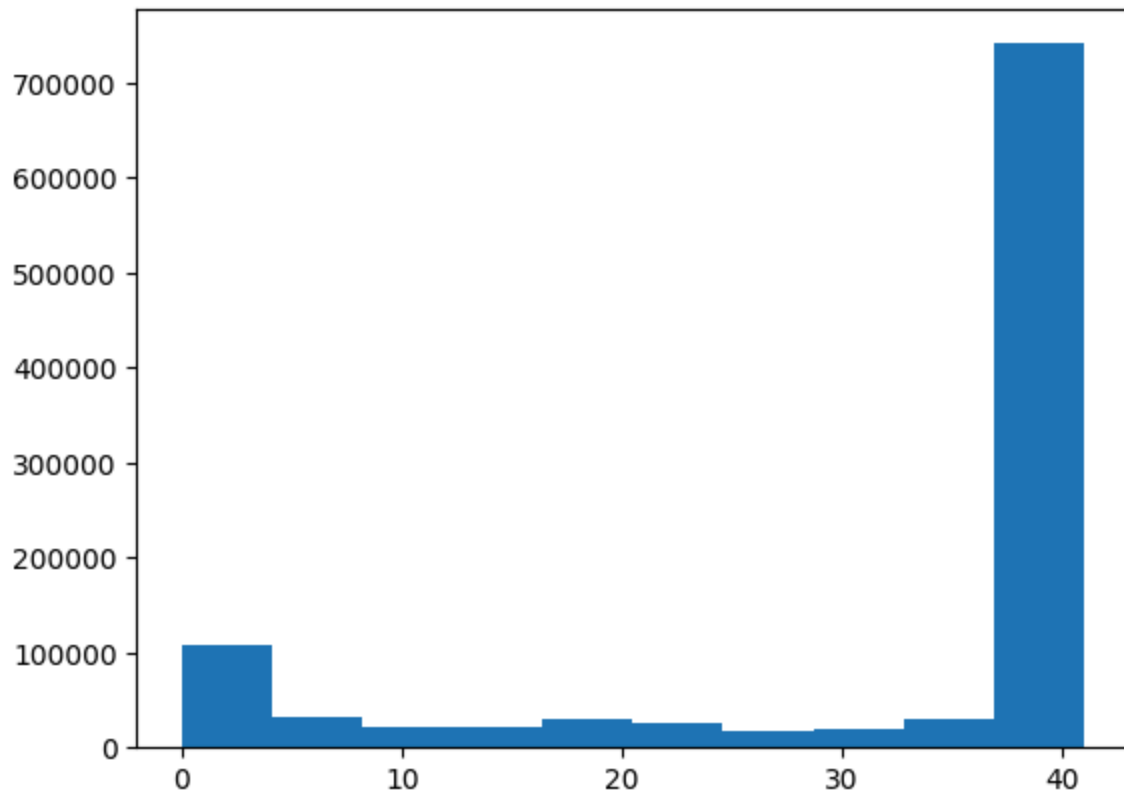
```
In [175... lbp = get_local_binary_pattern(img, radius=5)
plt.imshow(lbp, cmap="gray")
plt.axis("off")
plt.title("Local Binary Pattern (LBP)")
plt.show()
```


Local Binary Pattern (LBP)



```
In [176... bins = np.linspace(0, 41, 20)
plt.hist(lbp.ravel())
```

```
Out[176... (array([108607., 31761., 22364., 21738., 29735., 26642., 18362.,
        18842., 29673., 740852.]),
array([ 0. ,  4.1,  8.2, 12.3, 16.4, 20.5, 24.6, 28.7, 32.8, 36.9, 41. ]),
<BarContainer object of 10 artists>)
```



Another popular filter for image processing is the Gabor filter. The Gabor filters are a family of filters parametrized by orientation, frequency, bandwidth, etc. Let's generate one such filter.

Read more about Gabor filters: https://en.wikipedia.org/wiki/Gabor_filter and https://scikit-image.org/docs/stable/auto_examples/features_detection/plot_gabor.html

```
In [177... theta = 0
sigma = 6
frequency = 0.02

kernel = generate_gabor_kernel(theta, sigma, frequency)
gabor = get_gabor_features(img, kernel)
```

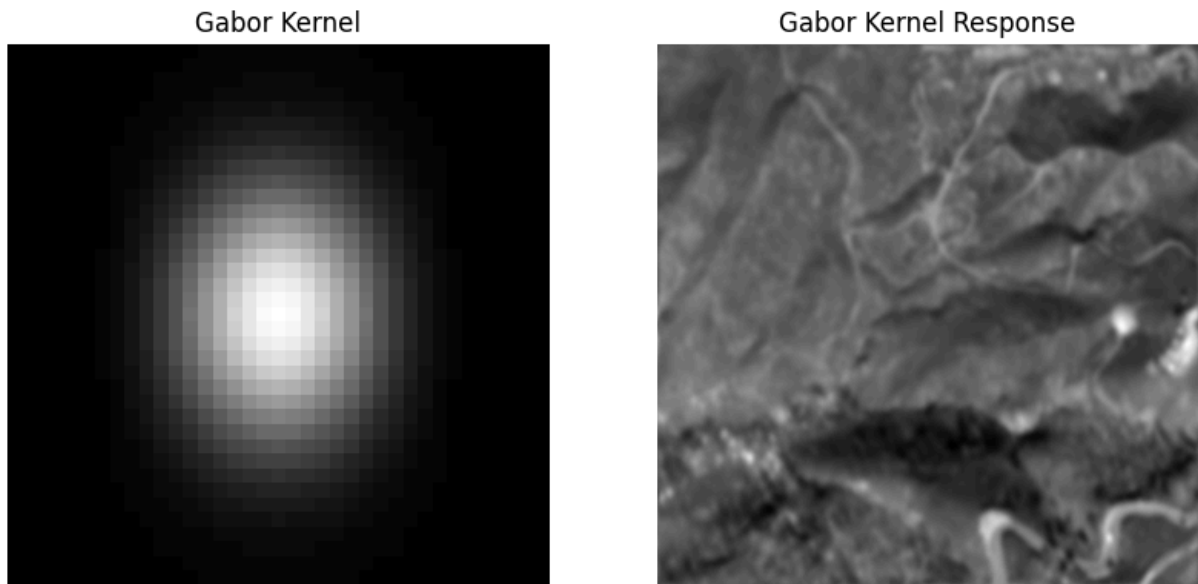
Let's visualize this filter (aka kernel) and the response of the image when we apply the filter. Concretely we are taking the convolution of the image with the filter. See [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing)) for more details.

```
In [178... fig, axs = plt.subplots(1, 2, figsize=(10, 5)) # 1 row, 2 columns

# Plot Gabor Kernel
axs[0].imshow(kernel, cmap="gray")
axs[0].axis("off") # Remove axis
axs[0].set_title("Gabor Kernel")

# Plot Gabor Kernel Response
axs[1].imshow(gabor, cmap="gray")
axs[1].axis("off") # Remove axis
```

```
axs[1].set_title("Gabor Kernel Response")  
plt.show()
```



Exploratory Data Analysis

The agency would like you to start performing some exploratory data analysis to build an initial understanding of the data. As part of the exploratory data analysis, the agency is interested in understanding certain aspects of the dataset better.

Feel free to add more cells as you see fit for your analysis.

EDA - Basic Statistics

Number of Images per Disaster Type

```
In [179]: for disaster, disaster_data in data.items():  
          print(f"{disaster} - {len(disaster_data['images'])} images")
```

```
socal-fire - 200 images  
midwest-flooding - 200 images
```

Distribution of Image Sizes in Each Dataset

We can compute the average, max, and min size of each image by dataset.

```
In [11]: disaster_image_shapes = {}  
for disaster, disaster_data in data.items():  
    width, height, _ = zip(*map(lambda img: img.shape, disaster_data['images']
```



```
disaster_image_shapes[disaster] = {'width_dist': width, 'height_dist': h
print(f"{disaster}\n\taverage -- width: {np.mean(width)} - height: {np.m
```

```
socal-fire
    average -- width: 1024.0 - height: 1024.0
    max -- width: 1024 - height: 1024
    min -- width: 1024 - height: 1024
midwest-flooding
    average -- width: 1024.0 - height: 1024.0
    max -- width: 1024 - height: 1024
    min -- width: 1024 - height: 1024
hurricane-matthew
    average -- width: 1024.0 - height: 1024.0
    max -- width: 1024 - height: 1024
    min -- width: 1024 - height: 1024
```

From the above, we see each image is the same size across datasets.

Note - Tiling Effects and Image Size

It would appear that while the images are the same sizes, this is due in part to them being resized with a black tiling effect. For example, check out image 24 from the socal fire dataset.

```
In [91]: socal_24 = data['socal-fire']['images'][24][37]
plt.imshow(socal_24)
plt.axis("off")
plt.margins(tight=True)
plt.title("Example Partially Blank Image (socal-fire-24)")
plt.savefig("../figures/example_tiled_image.png", format='png', bbox_inches=
plt.show()
```

Example Partially Blank Image (socal-fire-24)



To address this, we developed a function to perform a rectangular crop of the image to minimize the black portion.

```
In [1]: def crop_image(img):  
        crop = np.argwhere(np.sum(img, axis=2) != 0)  
        x_min, x_max = np.min(crop[:, 0]), np.max(crop[:, 0])  
        y_min, y_max = np.min(crop[:, 1]), np.max(crop[:, 1])  
        return img[x_min:x_max, y_min:y_max]
```

This is what the previous image looks like when cropped.

```
In [183]: plt.imshow(crop_image(socal_24))  
          plt.axis("off")  
          plt.title("Example Tiled Image - Cropped")  
          plt.show()
```

Example Tiled Image - Cropped



Let's get some baseline statistics on how prevalent this tiling is on images from each dataset

```
In [92]: def has_tiling(image):  
         return np.any(np.sum(image, axis=2) == 0)  
  
         print("socal fire tiling fraction: ")  
         print(f"{np.mean(list(map(has_tiling, data['socal-fire']['images']))):.1%}")  
  
         print("midwest flooding tiling fraction: ")  
         print(f"{np.mean(list(map(has_tiling, data['midwest-flooding']['images'])))}")  
  
         print("hurricane matthew tiling fraction: ")  
         print(f"{np.mean(list(map(has_tiling, data['hurricane-matthew']['images'])))}")
```

```
socal fire tiling fraction:  
23.0%  
midwest flooding tiling fraction:  
16.0%  
hurricane matthew tiling fraction:  
17.0%
```

Clearly, this issue is quite common, occurring in anywhere between 17% and 23% of images, depending on the specific dataset involved. Let's get a survey of the extent to which it's impacting the shapes of images in each of our datasets.

```
In [14]: disaster_images_cropped_data = []  
         for disaster, disaster_data in data.items():  
             width, height, _ = zip(*map(lambda img: crop_image(img).shape, disaster_  
                 for i in range(len(width)):
```

```
disaster_images_cropped_data.append((disaster, width[i], height[i]))
print(f"{disaster}\n\taverage -- width: {np.mean(width)} - height: {np.m
```

socal-fire

```
average -- width: 1011.55 - height: 957.9
max -- width: 1023 - height: 1023
min -- width: 126 - height: 323
```

midwest-flooding

```
average -- width: 999.555 - height: 999.625
max -- width: 1023 - height: 1023
min -- width: 132 - height: 228
```

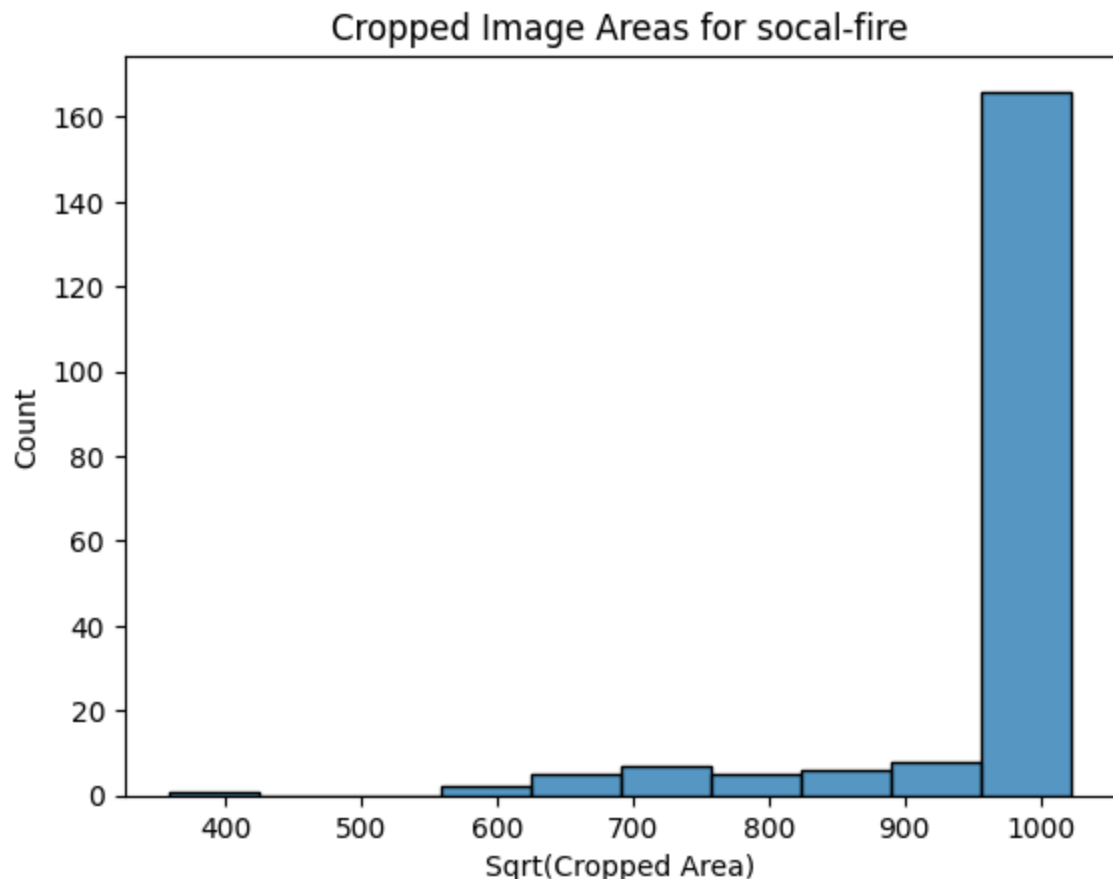
hurricane-matthew

```
average -- width: 994.45 - height: 1012.125
max -- width: 1023 - height: 1023
min -- width: 588 - height: 704
```

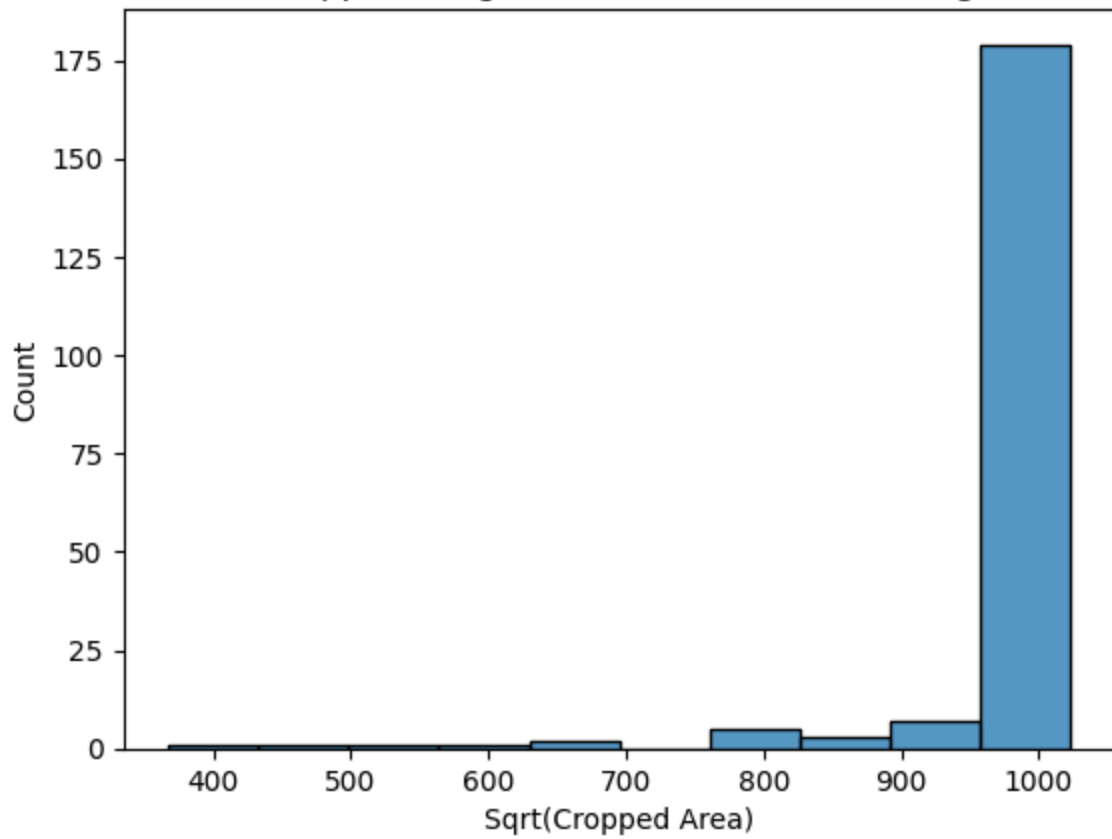
Let's also take a look at the distributions of image areas.

```
In [85]: image_shapes_df = pd.DataFrame.from_records(disaster_images_cropped_data, columns=['Disaster Label', 'Cropped Width', 'Cropped Height'])
image_shapes_df['Sqrt(Cropped Area)'] = np.sqrt(image_shapes_df['Cropped Width'] * image_shapes_df['Cropped Height'])

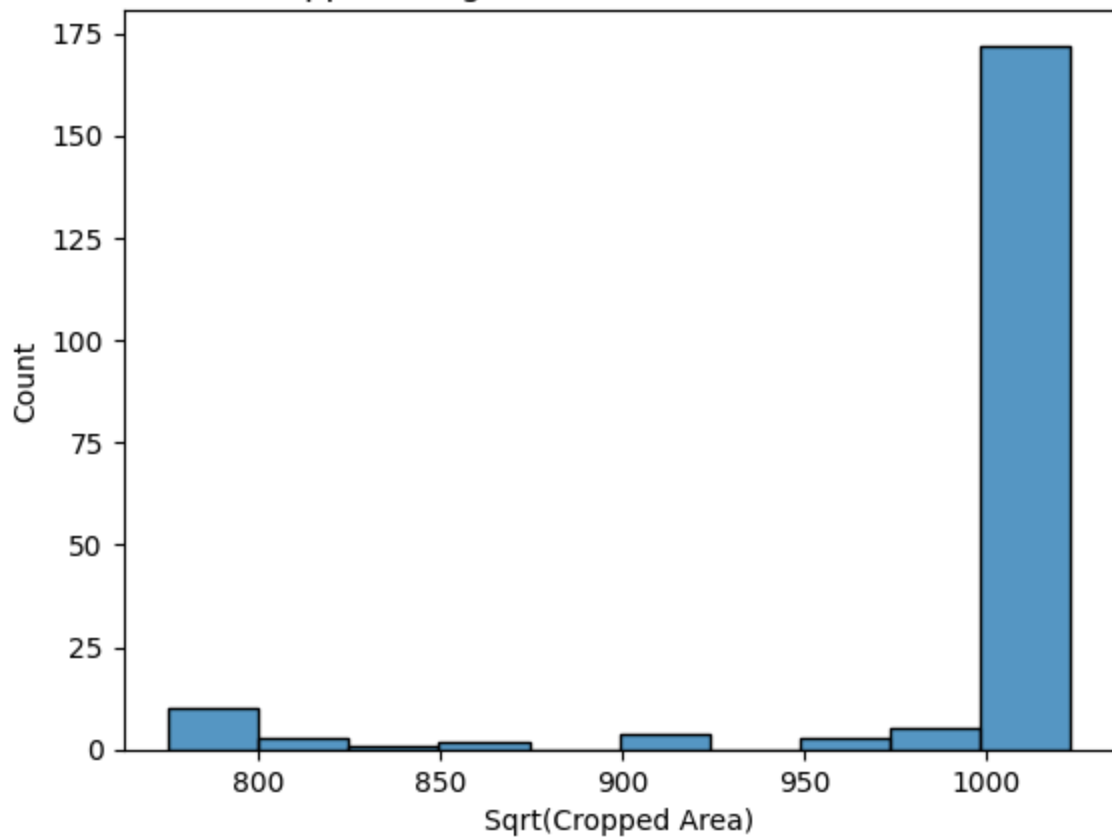
for i in range(3):
    fig = plt.figure()
    fig.add_axes(sns.histplot(image_shapes_df[image_shapes_df['Disaster Label'] == disaster_list[i]], 'Sqrt(Cropped Area)'))
    ax = fig.get_axes()[0]
    ax.set_title(f"Cropped Image Areas for {disaster_list[i]}", fontsize=12)
    plt.savefig(f"..../figures/{disaster_list[i]}-cropped-area-hist.png", format='png')
```



Cropped Image Areas for midwest-flooding



Cropped Image Areas for hurricane-matthew



We can see from the above that all datasets have a similar degree of impact by image. It would seem that perhaps the images associated with the hurricane are not as impacted as the other two datasets, as they have a higher average area of valid image overall. We can see from the histograms that this is due to them lacking as intense a left skew as the other two sets of image sizes.

Distribution of Damage Labels

EDA - Useful Image Features for Classification

The agency is looking for insights into useful image features for classifying images based on disaster type or damage level. You are tasked with verifying this and exploring whether color, edge, or texture features can effectively differentiate:

1. midwest-flooding from socal-fire images.
2. Damage levels 1 and 3 within the hurricane-matthew dataset.

Previous interns have found color information to be potentially useful.

Distribution of Color (RGB)

```
In [93]: # Helper function to compute average rgb of an image
def compute_average_rgb(image: np.ndarray):
    # Remove black pixels from tiling effect
    non_black_pixels = np.sum(np.einsum('ijk->ij', image > 0))

    # Return average image pixel rgb values
    return np.einsum('ijk->k', image.astype(int))/(non_black_pixels)
```

```
In [94]: rgb_distribution = {}
for disaster, disaster_data in data.items():
    average_colors = [compute_average_rgb(image) for image in data[disaster]]
    rgb_distribution[disaster] = list(zip(*average_colors))
```

```
In [95]: rgb_tuples = []
for disaster, color_distribution in rgb_distribution.items():
    for i in range(len(color_distribution[0])):
        colors = [color_distribution[j][i] for j in range(3)]
        rgb_tuples.append((disaster, *colors))
rgb_df = pd.DataFrame(rgb_tuples, columns=["disaster_label", "red", "green",
```

```
In [152... fig, axes = plt.subplots(1,3, dpi=100)
fig.set_figheight(5)
fig.set_figwidth(18)
```

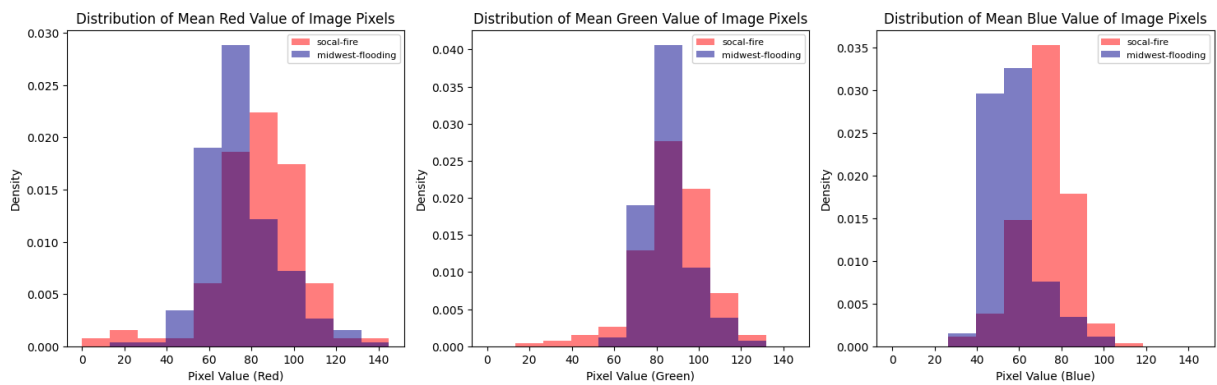


```

axes = axes.flatten()

for disaster, distribution in rgb_distribution.items():
    if disaster=='hurricane-matthew':
        continue
    for i in range(3):
        color = ['Red', 'Green', 'Blue'][i]
        ax = axes[i]
        # Plot histogram of color
        ax.hist(
            distribution[i],
            bins = np.linspace(0, 145, 12),
            label=disaster,
            alpha=0.5,
            density=True,
            color='red' if disaster=='social-fire' else 'darkblue',
        )
        ax.set_xlabel(f"Pixel Value ({color})")
        ax.set_ylabel(f"Density")
        ax.set_title(f"Distribution of Mean {color} Value of Image Pixels",
            ax.legend(fontsize=8);
plt.savefig("../figures/rgb-histograms.png")

```



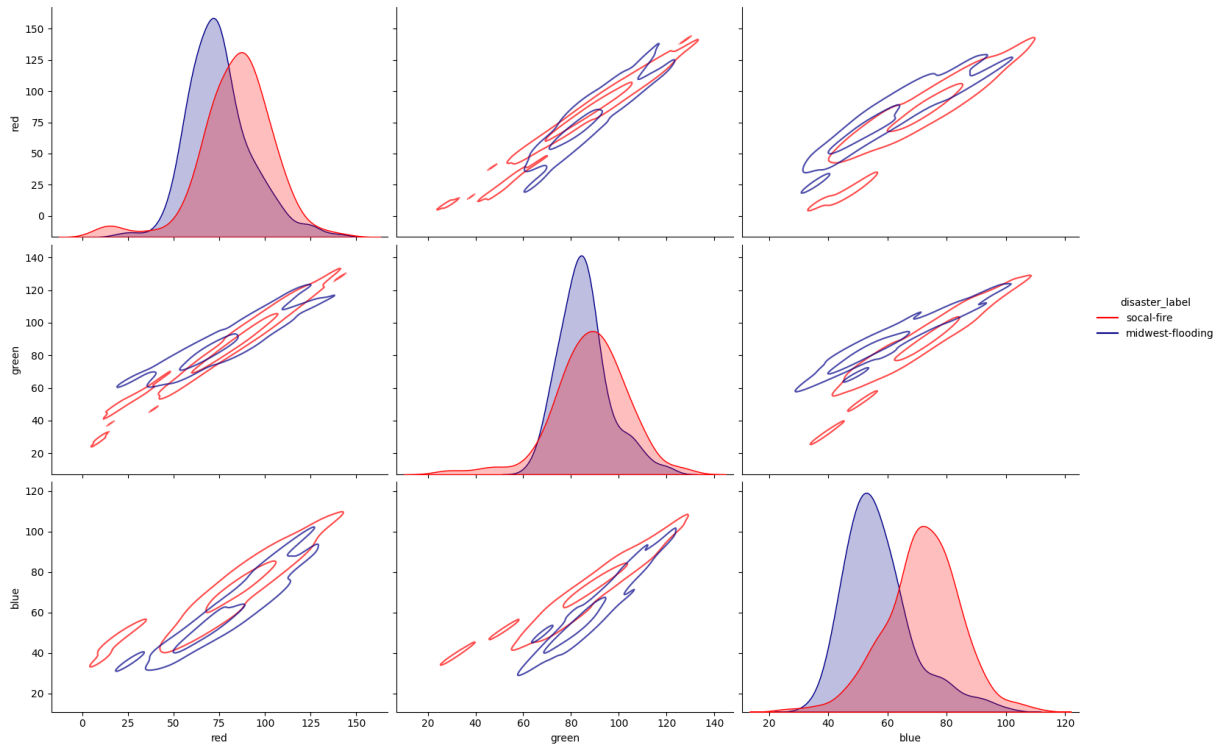
From the above, it's clear that the color blue does the best job of separating the two classes. We next check to see the degree of correlation, and thus mutual separability, between features.

```

In [158... pairplot = sns.pairplot(
    rgb_df[rgb_df['disaster_label']!='hurricane-matthew'],
    kind='kde',
    hue='disaster_label',
    height=4,
    aspect=1.25,
    palette=['red', 'darkblue'],
    plot_kws=dict(bw_adjust=1, levels=3, alpha=0.7),
)
pairplot.figure.suptitle("Pair Plot of Mean Red, Green, and Blue Image Value
pairplot.figure.subplots_adjust(top=0.85)
plt.savefig("../figures/rgb-pairplot.png")

```

Pair Plot of Mean Red, Green, and Blue Image Values by Disaster Type



From the above, it's clear that there is a pretty tight correlation between the color components. However, both green and red do add some additional separability when combined with blue.

How are the distributions "separable"?

The below are a non-exhaustive list of ways we can think about classifying these disaster images. You do not have to include all of them in your analysis.

1. midwest-flooding vs social-fire

Among the different disaster events, are there are distinctions in:

- image height, width, or overall size?
- RGB values?
- Sobel edges?
- Local Binary Patterns?
- Gabor kernels?

Gabor Kernels

```
In [189... from tqdm import tqdm
from joblib import Parallel, delayed
```

```

import plotly.express as px
from sklearn.decomposition import PCA
from sklearn.preprocessing import MinMaxScaler, RobustScaler
from sklearn.linear_model import LogisticRegression, LogisticRegressionCV
from scipy.spatial.distance import pdist
from feature_utils import generate_gabor_kernel, get_gabor_features

```

In [190... *### Helper functions for what we will do below ###*

```

def get_gabor_aggregate_features(image, kernel):
    gabor_filtered_image = get_gabor_features(image, kernel)
    var = np.var(gabor_filtered_image)
    mean = np.mean(gabor_filtered_image)
    return mean, var

def get_encoded_gabor_features(image, kernels) -> np.ndarray:
    feats = np.empty((len(kernels), 2), dtype=float)
    for i, kernel in enumerate(kernels):
        feats[i, 0], feats[i, 1] = get_gabor_aggregate_features(image, kernel)
    return feats

def get_features(image, disaster, kernels):
    return get_encoded_gabor_features(crop_image(image), kernels).flatten(),

```

Next, let's apply gabor transformations, and use the mean and variance of the pixels in the output to encode the texture of our image. To help avoid confounding from the blank regions, we will use the cropped images for this section.

```

In [ ]: # Generate our kernels – there will be 16 of these. This means each image
# will have 32 features (one per mean and one per variance)
kernels = []
for theta in range(4):
    theta = theta / 4.0 * np.pi
    for sigma in (1, 5):
        for frequency in (0.05, 0.25):
            kernel = generate_gabor_kernel(theta=theta, sigma=sigma, frequency=frequency)
            kernels.append(kernel)

# Get mean and variance of gabor filters applied to images for each of the kernels
# This takes *forever* when sigma=5, so we use joblib to parallelize the process
gabor_filter_feature_list = []
gabor_filter_label_list = []

for disaster, disaster_data in data.items():
    # Skip hurricane data for fire/flood classification
    if disaster == 'hurricane-matthew': continue

    parallel = Parallel(n_jobs=6, return_as="generator")
    for features, disaster in tqdm(parallel(delayed(get_features)(image, disaster, kernels)
        for image in disaster_data['images']):
            gabor_filter_feature_list.append(features)
            gabor_filter_label_list.append(disaster)

# Save our outputs as both numpy arrays and a dataframe
y, X = np.array(gabor_filter_label_list), np.vstack(gabor_filter_feature_list)

```

```
gb_features_df = pd.DataFrame(X[:, [i for i in range(X.shape[1])]], columns=
gb_features_df['disaster_label'] = y
```

Now that that's done, let's attempt to visualize the result using PCA. We'll take two approaches here, one raw, without scaling (this seems to be how the skimage tutorial compares theirs), and one using the sklearn robust scaler (there are outliers in this dataset).

```
In [197... scaler = RobustScaler()
gabor_pca = PCA(2)
scaled_pca = PCA(2)

visualization = gabor_pca.fit_transform(X)

scaled_X = scaler.fit_transform(X)
scaled_visualization = scaled_pca.fit_transform(scaled_X)

print(f"Gabor PCA explained variance: {np.sum(gabor_pca.explained_variance_r
print(f"Scaled Gabor PCA explained variance: {np.sum(scaled_pca.explained_va
```

Gabor PCA explained variance: 100.00%

Scaled Gabor PCA explained variance: 69.46%

Let's get a ballpark for how good the separation is in the scaled model with a simple logistic regression model.

```
In [199... # Unscaled
lr_model = LogisticRegression(fit_intercept=True)
lr_model.fit(visualization, y)
score = lr_model.score(visualization, y)
print(f"Line of best separation for raw gabor pca achieves accuracy of {score}

# Scaled
lr_model = LogisticRegression(fit_intercept=True)
lr_model.fit(scaled_visualization[:, 0:2], y)
score = lr_model.score(scaled_visualization[:, 0:2], y)
print(f"Line of best separation for scaled gabor achieves accuracy of {score}
```

Line of best separation for raw gabor pca achieves accuracy of 63%.

Line of best separation for scaled gabor achieves accuracy of 83%.

This is quite encouraging! While by no means the same as testing it against a validation set of data, the fact that such a simple model can achieve this degree of separation is very promising. It also confirms that scaling is important here. Some of the gabor kernels simply have higher average intensities than others, without conveying more information.

The next step is to visualize the distributions of the two kernels over the principle components.

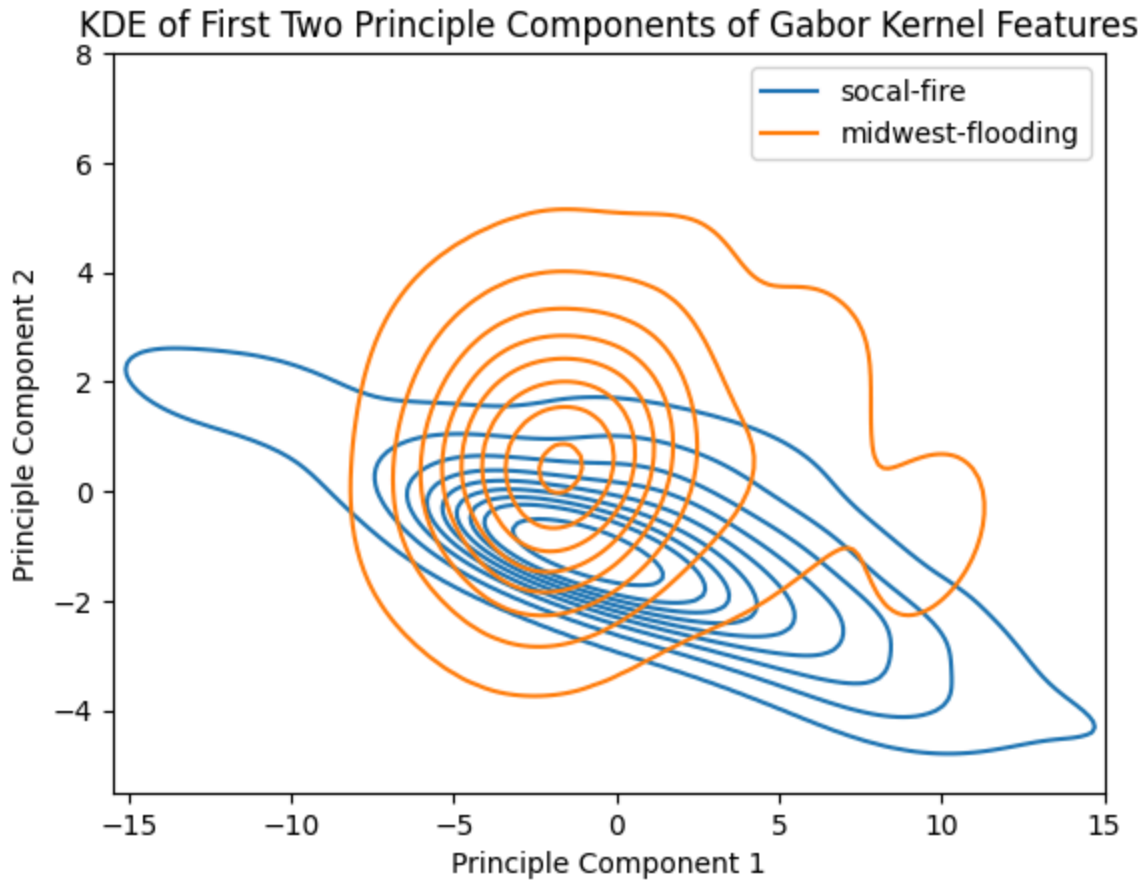
```
In [200... kde_plot = sns.kdeplot(
    data=pd.DataFrame(np.vstack([scaled_visualization[:, 0], scaled_visualiz
    x="Principle Component 1",
    y="Principle Component 2",
```

```

    hue=y,
)

plt.xlim((-15.5, 15))
plt.ylim((-5.5, 8))
plt.title('KDE of First Two Principle Components of Gabor Kernel Features')
plt.savefig("../figures/gabor_kernel_kde.png");

```



As suggested by the accuracy of the linear regression model, the distributions of the two classes over the first two principle components are decently separated here. We can also see what might be a group of outliers in the fire dataset around the region from -15 to -10 of the first PC.

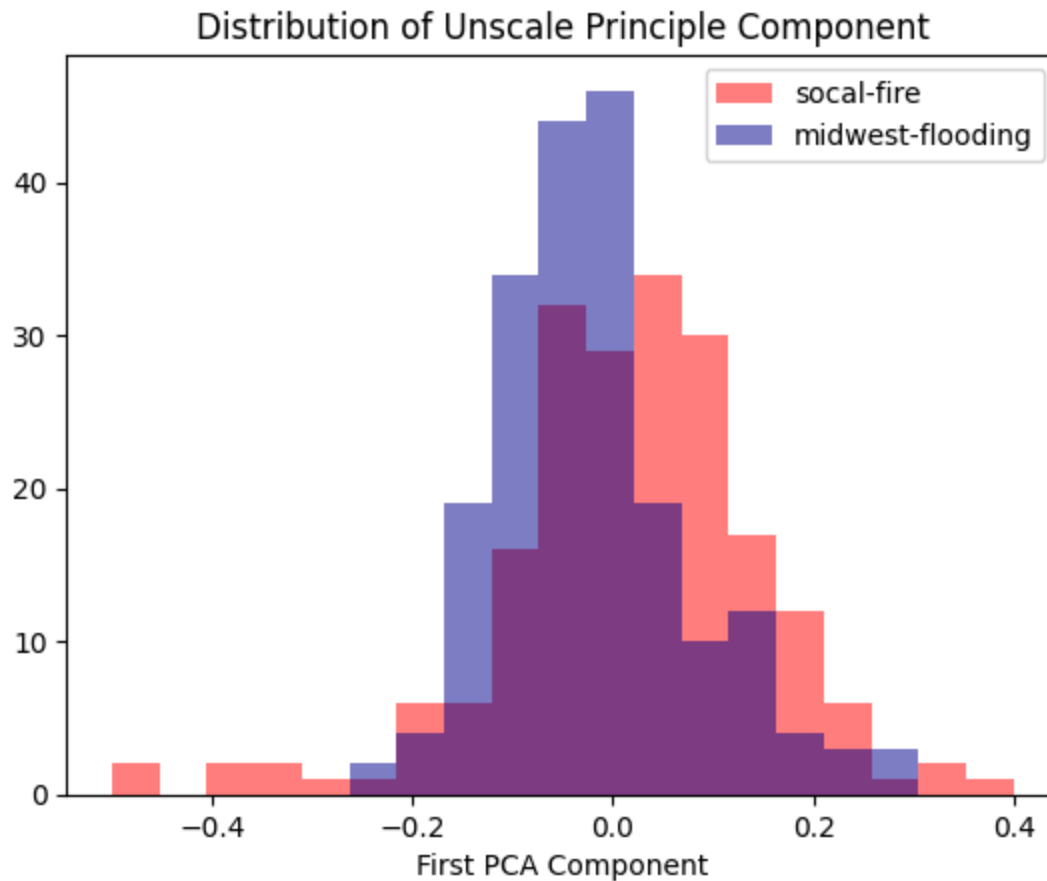
Let's also plot the principle component of the unscaled data.

In [206...

```

bins = np.linspace(-0.5, 0.4, 20)
plt.hist(visualization[y=='socal-fire', 0], bins=bins, label='socal-fire', a
plt.hist(visualization[y=='midwest-flooding', 0], bins=bins, label='midwest
plt.xlabel("First PCA Component")
plt.title("Distribution of Unscale Principle Component")
plt.legend()
plt.savefig("../figures/unscaled_pc1_hist.png")

```



There appears to be a small, secondary peak for very low PC1 values of Socal Fire. Let's take a look at the images that are in this bump.

```
In [ ]: outlier_image_idx = np.argwhere((visualization[:, 0] < -0.2) & (y=='socal-f
outlier_images = [ crop_image(data['socal-fire']['images'][i]) for i in outl

fig, axes = plt.subplots(2, 4, figsize=(10,5))

fig.suptitle("Outlier socal-fire Images in PC1 of Gabor Kernel Image Feature

for ax, image, image_number in zip(axes.flatten(), outlier_images, outlier_i
    ax.imshow(image)
    ax.set_title(f"Image-{image_number} of socal-fire")
    ax.axis("off")
plt.savefig("../figures/gabor_outlier_images.png");
```

Outlier social-fire Images in PC1 of Gabor Kernel Image Features

Image-4 of social-fire



Image-27 of social-fire

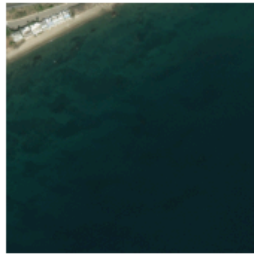


Image-109 of social-fire

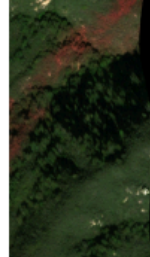


Image-115 of social-fire



Image-131 of social-fire

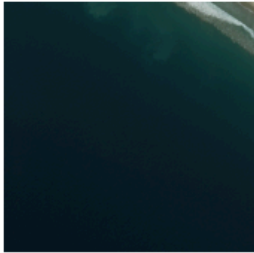


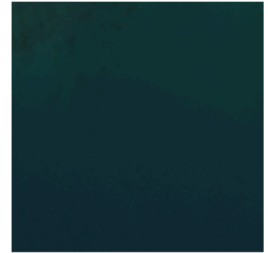
Image-138 of social-fire



Image-174 of social-fire



Image-176 of social-fire



Interesting! With the exception of Image-109, these are all images consisting mostly of the ocean. Perhaps we should attempt to identify these separately and filter them out first before developing our primary model. I suspect that a simple average color/variance check will be sufficient to do so reliably.

Conclusion - Gabor Kernels

Given the above histogram, it appears the features from gabor kernels can at least partially separate the images of fires vs. flooding. It also proved useful for identify outliers that I otherwise may not have noticed.